BÀI THỰC HÀNH CTDL – CẤU TRÚC ĐỒ THỊ (GRAPH)

Lab 07 · Python 3.8+ · 10 bài tập từ cơ bản đến nâng cao

Sau khi làm việc với cây (một dạng đồ thị đặc biệt: liên thông và không có chu trình), hôm nay chúng ta bước sang cấu trúc phi tuyến tổng quát nhất: **đồ thị (graph)**. Đồ thị dùng để mô hình hóa mọi thứ có quan hệ qua lại như bản đồ giao thông, mạng xã hội, mạng máy tính, lịch học các môn tiên quyết hay mê cung.

Bài thực hành đi từ cách biểu diễn đồ thị, hai phép duyệt DFS/BFS, các bài toán về liên thông và chu trình, sắp xếp tô-pô, đường đi ngắn nhất (Dijkstra, Bellman-Ford), cây khung nhỏ nhất (Kruskal) đến phát hiện cầu và khớp trong mạng.

Mục Đích và Yêu Cầu Chung

🎯 Mục đích

- **Nắm vững khái niệm:** đỉnh, cạnh, bậc, đường đi, chu trình, liên thông, đồ thị có hướng/vô hướng, có trọng số.
- **Biểu diễn đồ thị:** chọn đúng giữa ma trận kề và danh sách kề theo bài toán.
- **Thành thạo duyệt đồ thị:** DFS và BFS, và biết khi nào dùng cái nào.
- **Cài đặt các thuật toán kinh điển:** tô-pô, Dijkstra, Bellman-Ford, Kruskal, tìm cầu và khớp.
- **Giải quyết bài toán thực tế:** tìm đường, xếp lịch học, thiết kế mạng, đánh giá độ tin cậy của mạng.

📋 Yêu cầu chung cho tất cả bài tập

- **Môi trường:** Console Application bằng Python 3.8 trở lên.
- **Code sạch:** đặt tên theo chuẩn PEP 8 (snake_case), có chú thích ở các đoạn logic quan trọng.
- **Tránh lỗi quen thuộc:** luôn dùng tập `visited` để không lặp vô hạn khi đồ thị có chu trình; kiểm tra đỉnh không tồn tại trước khi truy cập.
- **Kiểm thử:** mỗi bài chạy với ít nhất một đồ thị mẫu, kể cả trường hợp biên (đồ thị rỗng, một đỉnh, đồ thị không liên thông, đồ thị có chu trình).
- **Học sâu với AI:** tận dụng AI để khám phá các khái niệm như độ phức tạp O(V+E), đồ thị thưa/dày, nhưng phải tự chạy và hiểu code trước khi nộp.

Bài tập 1: Biểu diễn đồ thị – Ma trận kề và danh sách kề – Mức độ: Dễ

Bài đầu tiên giúp bạn làm quen với hai cách lưu đồ thị trong bộ nhớ và so sánh ưu nhược điểm của chúng.

Yêu cầu chi tiết:

- Tạo lớp `Graph` hỗ trợ cả đồ thị vô hướng và có hướng, có trọng số (mặc định trọng số = 1).
- Lưu đồng thời bằng **ma trận kề** và **danh sách kề**.

- Viết các phương thức `add_edge(u, v, weight)`, `has_edge(u, v)`, `degree(v)`, `in_degree(v)`.
- In ma trận kề và danh sách kề của đồ thị mẫu có ít nhất 5 đỉnh, 6 cạnh.
- Tạo thêm một đồ thị có hướng và so sánh bậc vào, bậc ra của từng đỉnh.

💡 **Gợi ý:**

- Ma trận kề: kiểm tra cạnh O(1) nhưng tốn $O(V^2)$ bộ nhớ. Danh sách kề: tốn O(V+E), duyệt các đỉnh kề nhanh.
- Đồ thị vô hướng: thêm cạnh phải cập nhật cả hai chiều (u→v và v→u).
- Dùng `dict` ánh xạ tên đỉnh sang chỉ số hàng/cột của ma trận.

💻 **Code mẫu:**

```
class Graph:
    """Đồ thị lưu đồng thời bằng ma trận kề và danh sách kề."""

    def __init__(self, vertices, directed=False):
        self.vertices = list(vertices)
        self.index = {v: i for i, v in enumerate(self.vertices)}
        self.directed = directed
        n = len(self.vertices)
        self.matrix = [[0] * n for _ in range(n)]
        self.adj = {v: [] for v in self.vertices}

    def add_edge(self, u, v, weight=1):
        i, j = self.index[u], self.index[v]
        self.matrix[i][j] = weight
        self.adj[u].append((v, weight))
        if not self.directed:  # Vô hướng: cập nhật cả chiều ngược lại
            self.matrix[j][i] = weight
            self.adj[v].append((u, weight))

    def has_edge(self, u, v):
        return self.matrix[self.index[u]][self.index[v]] != 0

    def degree(self, v):  # Vô hướng: bậc; có hướng: bậc ra
        return len(self.adj[v])

    def in_degree(self, v):
        j = self.index[v]
        return sum(1 for row in self.matrix if row[j] != 0)

    def print_matrix(self):
        print('   ', ' '.join(self.vertices))
        for v, row in zip(self.vertices, self.matrix):
            print(v, ' ', ' '.join(str(x) for x in row))

    def print_adj(self):
        for v in self.vertices:
            print(v, '->', [u for u, _ in self.adj[v]])


def main():
    g = Graph(['A', 'B', 'C', 'D', 'E'])
```

```
    for u, v in [('A', 'B'), ('A', 'C'), ('B', 'D'), ('C', 'D'), ('D', 'E'), ('B',
'C')]:
        g.add_edge(u, v)
    g.print_matrix()
    g.print_adj()
    print('Có cạnh A-D?', g.has_edge('A', 'D'))
    print('Bậc của D:', g.degree('D'))

    d = Graph(['X', 'Y', 'Z'], directed=True)
    d.add_edge('X', 'Y')
    d.add_edge('X', 'Z')
    d.add_edge('Y', 'Z')
    for v in d.vertices:
        print(f'{v}: bậc ra = {d.degree(v)}, bậc vào = {d.in_degree(v)}')


if __name__ == '__main__':
    main()
```

🤖 **Vận dụng AI để gia tăng trải nghiệm:**

- **Hỏi về lý thuyết:** Hỏi AI: *Khi nào nên dùng ma trận kề, khi nào dùng danh sách kề? Thế nào là đồ thị thưa và đồ thị dày?*
- **Khám phá thêm:** Hỏi AI: *Cách biểu diễn đồ thị bằng danh sách cạnh (edge list) phù hợp với thuật toán nào?*

Bài tập 2: Duyệt đồ thị theo chiều sâu (DFS) – Mức độ: Dễ

DFS đi sâu hết mức có thể theo một nhánh rồi mới quay lui. Khác với cây, đồ thị có thể có chu trình nên bắt buộc phải đánh dấu đỉnh đã thăm.

Yêu cầu chi tiết:

- Biểu diễn đồ thị bằng `dict` danh sách kề (đỉnh → list các đỉnh kề).
- Viết `dfs_recursive(graph, start)` bằng đệ quy và `dfs_iterative(graph, start)` bằng stack tự quản lý.
- Cả hai hàm trả về list các đỉnh theo thứ tự thăm; kiểm tra hai kết quả trùng nhau.
- Viết thêm `has_path(graph, u, v)` kiểm tra có đường đi từ u đến v hay không.

💡 **Gợi ý:**

- Điều kiện dừng: đỉnh đã nằm trong `visited`.
- Với stack, đẩy các đỉnh kề theo thứ tự **ngược** (`reversed`) để thứ tự thăm giống bản đệ quy.
- Độ phức tạp thời gian O(V + E).

💻 **Code mẫu:**

```
def dfs_recursive(graph, start, visited=None, order=None):
    if visited is None:
        visited, order = set(), []
    visited.add(start)
    order.append(start)
    for v in graph[start]:
```

```
        if v not in visited:
            dfs_recursive(graph, v, visited, order)
    return order


def dfs_iterative(graph, start):
    visited, order = set(), []
    stack = [start]
    while stack:
        u = stack.pop()
        if u in visited:
            continue
        visited.add(u)
        order.append(u)
        for v in reversed(graph[u]):  # Đảo để thứ tự giống đệ quy
            if v not in visited:
                stack.append(v)
    return order


def has_path(graph, u, v):
    return v in dfs_iterative(graph, u)


def main():
    graph = {
        'A': ['B', 'C'],
        'B': ['A', 'D', 'E'],
        'C': ['A', 'F'],
        'D': ['B'],
        'E': ['B', 'F'],
        'F': ['C', 'E'],
    }
    r1 = dfs_recursive(graph, 'A')
    r2 = dfs_iterative(graph, 'A')
    print('DFS đệ quy :', r1)
    print('DFS stack  :', r2)
    print('Hai kết quả trùng nhau?', r1 == r2)
    print('Có đường D -> F?', has_path(graph, 'D', 'F'))


if __name__ == '__main__':
    main()
```

🤖 **Vận dụng AI để gia tăng trải nghiệm:**

- **Hỏi về lý thuyết:** Hỏi AI: *Vì sao DFS đệ quy có thể gây RecursionError với đồ thị lớn? Giới hạn đệ quy của Python là bao nhiêu?*
- **Khám phá thêm:** Hỏi AI: *DFS ghi lại thời điểm vào/ra (discovery/finish time) dùng để làm gì?*

Bài tập 3: Duyệt theo chiều rộng (BFS) và đường đi ngắn nhất – Mức độ: Dễ – Trung bình

BFS loang từ đỉnh xuất phát theo từng lớp. Trên đồ thị không trọng số, BFS cho đường đi ít cạnh nhất.

Yêu cầu chi tiết:

- Viết `bfs(graph, start)` trả về thứ tự thăm.
- Viết `shortest_path(graph, start, goal)` trả về danh sách đỉnh trên đường đi ngắn nhất (hoặc `None` nếu không có đường).
- Viết `distances(graph, start)` trả về `dict` khoảng cách (số cạnh) từ `start` đến mọi đỉnh.
- Kiểm thử với cặp đỉnh có đường đi, cặp đỉnh ở hai thành phần khác nhau và trường hợp `start == goal`.

💡 **Gợi ý:**

- Dùng `collections.deque` với `popleft()` (O(1)).
- Lưu `parent[v] = u` khi lần đầu gặp v, rồi truy ngược từ `goal` về `start` để dựng đường đi.
- Đánh dấu đã thăm **ngay khi đưa vào hàng đợi** để mỗi đỉnh chỉ vào queue một lần.

💻 **Code mẫu:**

```
from collections import deque


def bfs(graph, start):
    visited = {start}
    order = []
    queue = deque([start])
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in graph[u]:
            if v not in visited:
                visited.add(v)
                queue.append(v)
    return order


def shortest_path(graph, start, goal):
    parent = {start: None}
    queue = deque([start])
    while queue:
        u = queue.popleft()
        if u == goal:
            break
        for v in graph[u]:
            if v not in parent:
                parent[v] = u
                queue.append(v)
    if goal not in parent:
        return None  # Không có đường đi
    path = []
    cur = goal
    while cur is not None:  # Truy ngược về start
        path.append(cur)
        cur = parent[cur]
    return path[::-1]
```

```
def distances(graph, start):
    dist = {start: 0}
    queue = deque([start])
    while queue:
        u = queue.popleft()
        for v in graph[u]:
            if v not in dist:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist


def main():
    graph = {
        'A': ['B', 'C'],
        'B': ['A', 'D', 'E'],
        'C': ['A', 'F'],
        'D': ['B'],
        'E': ['B', 'F'],
        'F': ['C', 'E'],
        'G': [],  # Đỉnh cô lập
    }
    print('BFS từ A:', bfs(graph, 'A'))
    print('Đường A -> F:', shortest_path(graph, 'A', 'F'))
    print('Đường A -> G:', shortest_path(graph, 'A', 'G'))
    print('Khoảng cách từ A:', distances(graph, 'A'))


if __name__ == '__main__':
    main()
```

🤖 **Vận dụng AI để gia tăng trải nghiệm:**

- **Hỏi về lý thuyết:** Hỏi AI: *Vì sao BFS đảm bảo đường đi ngắn nhất (theo số cạnh) còn DFS thì không?*
- **Nâng cao:** Hỏi AI: *BFS đa nguồn (multi-source BFS) là gì? Áp dụng thế nào cho bài toán lan truyền lửa/nước trên lưới?*

Bài tập 4: Thành phần liên thông và đếm hòn đảo – Mức độ: Trung bình

Một đồ thị có thể gồm nhiều mảnh rời nhau. Mỗi mảnh là một thành phần liên thông. Cùng ý tưởng này giải được bài toán đếm đảo trên bản đồ dạng lưới.

Yêu cầu chi tiết:

- Viết `connected_components(graph)` trả về danh sách các thành phần liên thông (mỗi thành phần là một list đỉnh) của đồ thị vô hướng.
- Viết `is_connected(graph)` kiểm tra đồ thị có liên thông hay không.
- Viết `count_islands(grid)`: lưới gồm các ký tự `'1'` (đất) và `'0'` (nước); đếm số đảo, hai ô đất kề nhau theo 4 hướng (trên, dưới, trái, phải) thì thuộc cùng một đảo.
- Thử với đồ thị rỗng, đồ thị một đỉnh và lưới toàn nước.

💡 **Gợi ý:**

- Với mỗi đỉnh chưa thăm, chạy DFS/BFS một lần và gom các đỉnh tìm được; số lần chạy chính là số thành phần.
- Lưới là đồ thị ngầm định: đỉnh là ô `(r, c)`, đỉnh kề là 4 ô lân cận còn nằm trong lưới.
- Dùng stack thay vì đệ quy để an toàn với lưới lớn.

💻 **Code mẫu:**

```
def connected_components(graph):
    visited = set()
    components = []
    for s in graph:
        if s in visited:
            continue
        comp = []
        stack = [s]
        visited.add(s)
        while stack:
            u = stack.pop()
            comp.append(u)
            for v in graph[u]:
                if v not in visited:
                    visited.add(v)
                    stack.append(v)
        components.append(comp)
    return components


def is_connected(graph):
    return len(connected_components(graph)) <= 1


def count_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    visited = set()
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1' and (r, c) not in visited:
                count += 1  # Gặp đảo mới -> loang hết đảo này
                visited.add((r, c))
                stack = [(r, c)]
                while stack:
                    x, y = stack.pop()
                    for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                        nx, ny = x + dx, y + dy
                        if (0 <= nx < rows and 0 <= ny < cols
                                and grid[nx][ny] == '1' and (nx, ny) not in visited):
                            visited.add((nx, ny))
                            stack.append((nx, ny))
    return count
```

```
def main():
    graph = {'A': ['B'], 'B': ['A'], 'C': ['D'], 'D': ['C'], 'E': []}
    comps = connected_components(graph)
    print('Các thành phần liên thông:', comps)
    print('Số thành phần:', len(comps), '| Liên thông?', is_connected(graph))

    grid = ['11000',
            '11000',
            '00100',
            '00011']
    print('Số đảo:', count_islands(grid))  # 3


if __name__ == '__main__':
    main()
```

🤖 **Vận dụng AI để gia tăng trải nghiệm:**

- **Hỏi về lý thuyết:** Hỏi AI: *Khác nhau giữa liên thông yếu và liên thông mạnh trong đồ thị có hướng là gì?*
- **Khám phá thêm:** Hỏi AI: *Tìm diện tích đảo lớn nhất trong lưới và xử lý ô lân cận theo 8 hướng.*

Bài tập 5: Phát hiện chu trình – Mức độ: Trung bình

Cây là đồ thị không có chu trình. Phát hiện chu trình là bước kiểm tra nền tảng, chẳng hạn để biết một đồ thị có phải là cây hay một hệ phụ thuộc có bị vòng lặp không. Cách làm khác nhau giữa đồ thị vô hướng và có hướng.

Yêu cầu chi tiết:

- Viết `has_cycle_undirected(graph)` cho đồ thị vô hướng.
- Viết `has_cycle_directed(graph)` cho đồ thị có hướng bằng ba trạng thái đỉnh: chưa thăm, đang thăm, đã xong.
- Viết `is_tree(graph)`: đồ thị vô hướng là cây khi liên thông và không có chu trình.
- Kiểm thử với: tam giác (có chu trình), đường thẳng (không), đồ thị có hướng A→B→C→A (có), đồ thị có hướng kiểu kim cương (không có chu trình).

💡 **Gợi ý:**

- Vô hướng: khi duyệt gặp đỉnh đã thăm **không phải là cha** của đỉnh hiện tại thì có chu trình.
- Có hướng: gặp lại đỉnh đang ở trạng thái *đang thăm* (đang nằm trên đường đi DFS hiện tại) nghĩa là có chu trình. Gặp đỉnh *đã xong* thì không sao.
- Không dùng cách kiểm tra "đã thăm là có chu trình" cho đồ thị có hướng, vì sẽ báo sai với đồ thị kim cương.

💻 **Code mẫu:**

```
def has_cycle_undirected(graph):
    visited = set()
```

```
    def dfs(u, parent):
        visited.add(u)
        for v in graph[u]:
            if v not in visited:
                if dfs(v, u):
                    return True
            elif v != parent:  # Đã thăm và không phải cha -> chu trình
                return True
        return False

    return any(s not in visited and dfs(s, None) for s in graph)


def has_cycle_directed(graph):
    WHITE, GRAY, BLACK = 0, 1, 2  # Chưa thăm / đang thăm / đã xong
    color = {u: WHITE for u in graph}

    def dfs(u):
        color[u] = GRAY
        for v in graph[u]:
            if color[v] == GRAY:  # Quay về đỉnh đang trên đường đi
                return True
            if color[v] == WHITE and dfs(v):
                return True
        color[u] = BLACK
        return False

    return any(color[u] == WHITE and dfs(u) for u in graph)


def is_tree(graph):
    if not graph:
        return False
    start = next(iter(graph))
    visited, stack = {start}, [start]
    while stack:
        u = stack.pop()
        for v in graph[u]:
            if v not in visited:
                visited.add(v)
                stack.append(v)
    return len(visited) == len(graph) and not has_cycle_undirected(graph)


def main():
    triangle = {'A': ['B', 'C'], 'B': ['A', 'C'], 'C': ['A', 'B']}
    line = {'A': ['B'], 'B': ['A', 'C'], 'C': ['B']}
    print('Tam giác có chu trình?', has_cycle_undirected(triangle))  # True
    print('Đường thẳng có chu trình?', has_cycle_undirected(line))  # False
    print('Đường thẳng là cây?', is_tree(line))  # True

    loop = {'A': ['B'], 'B': ['C'], 'C': ['A']}
    diamond = {'A': ['B', 'C'], 'B': ['D'], 'C': ['D'], 'D': []}
    print('A->B->C->A có chu trình?', has_cycle_directed(loop))  # True
    print('Kim cương có chu trình?', has_cycle_directed(diamond))  # False
```

```
if __name__ == '__main__':
    main()
```

🤖 **Vận dụng AI để gia tăng trải nghiệm:**

- **Xử lý lỗi:** Cố tình dùng thuật toán của đồ thị vô hướng cho đồ thị có hướng rồi hỏi AI: *Hãy chỉ ra đồ thị khiến hàm này trả về sai.*
- **Khám phá thêm:** Hỏi AI: *Cấu trúc Union-Find phát hiện chu trình trong đồ thị vô hướng như thế nào?*

Bài tập 6: Sắp xếp tô-pô – Xếp lịch môn học – Mức độ: Trung bình – Khó

Trong đồ thị có hướng không chu trình (DAG), sắp xếp tô-pô cho một thứ tự mà mọi cạnh u→v đều có u đứng trước v. Ứng dụng kinh điển: thứ tự học các môn tiên quyết, thứ tự biên dịch các module, thứ tự thực hiện công việc.

Yêu cầu chi tiết:

- Cài đặt `topological_sort(graph)` theo thuật toán Kahn (dùng bậc vào và hàng đợi).
- Nếu đồ thị có chu trình, hàm trả về `None` kèm thông báo không thể xếp lịch.
- Dựng đồ thị các môn học (ít nhất 6 môn, có quan hệ tiên quyết), in một thứ tự học hợp lệ.
- Thêm một quan hệ tiên quyết gây vòng lặp và kiểm tra hàm phát hiện được.
- Viết hàm `is_valid_order(graph, order)` kiểm tra thứ tự có thỏa mọi ràng buộc hay không.

💡 **Gợi ý:**

- Tính bậc vào của mọi đỉnh, đưa các đỉnh bậc vào 0 vào hàng đợi.
- Mỗi lần lấy một đỉnh ra, giảm bậc vào của các đỉnh kề; đỉnh nào về 0 thì đưa vào hàng đợi.
- Nếu số đỉnh đã xếp nhỏ hơn tổng số đỉnh thì còn chu trình.

💻 **Code mẫu:**

```
from collections import deque


def topological_sort(graph):
    in_degree = {u: 0 for u in graph}
    for u in graph:
        for v in graph[u]:
            in_degree[v] += 1
    queue = deque(u for u in graph if in_degree[u] == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in graph[u]:
            in_degree[v] -= 1
            if in_degree[v] == 0:
                queue.append(v)
    return order if len(order) == len(graph) else None  # None: có chu trình
```

```
def is_valid_order(graph, order):
    position = {u: i for i, u in enumerate(order)}
    return all(position[u] < position[v] for u in graph for v in graph[u])


def main():
    # Cạnh u -> v nghĩa là: phải học u trước khi học v
    courses = {
        'Lap trinh co ban': ['CTDL', 'Co so du lieu'],
        'Toan roi rac': ['CTDL', 'Do thi'],
        'CTDL': ['Giai thuat', 'Do thi'],
        'Co so du lieu': ['Phan mem ung dung'],
        'Giai thuat': ['Phan mem ung dung'],
        'Do thi': [],
        'Phan mem ung dung': [],
    }
    order = topological_sort(courses)
    print('Thứ tự học:', ' -> '.join(order))
    print('Hợp lệ?', is_valid_order(courses, order))

    courses['Phan mem ung dung'].append('Lap trinh co ban')  # Tạo vòng lặp
    if topological_sort(courses) is None:
        print('Có vòng lặp tiên quyết, không thể xếp lịch!')


if __name__ == '__main__':
    main()
```

🤖 **Vận dụng AI để gia tăng trải nghiệm:**

- **Khám phá thêm:** Hỏi AI: *Sắp xếp tô-pô bằng DFS (thứ tự hoàn thành đảo ngược) khác thuật toán Kahn thế nào?*
- **Nâng cao:** Hỏi AI: *Làm sao in tất cả các thứ tự tô-pô hợp lệ, hoặc xếp học kỳ sao cho số học kỳ ít nhất?*

Bài tập 7: Đường đi ngắn nhất có trọng số – Thuật toán Dijkstra – Mức độ: Khó

Khi các cạnh có trọng số (độ dài đường, thời gian di chuyển, chi phí), BFS không còn đúng nữa. Dijkstra luôn mở rộng đỉnh có khoảng cách tạm thời nhỏ nhất, dùng hàng đợi ưu tiên (heap). Thuật toán yêu cầu mọi trọng số không âm.

Yêu cầu chi tiết:

- Biểu diễn đồ thị có trọng số bằng dict: đỉnh → list các cặp (đỉnh kề, trọng số).
- Cài đặt `dijkstra(graph, source)` dùng `heapq`, trả về hai dict: `dist` (khoảng cách ngắn nhất) và `parent` (để dựng lại đường đi).
- Viết `get_path(parent, target)` dựng đường đi từ nguồn đến đích.
- Dựng bản đồ mini gồm ít nhất 7 địa điểm (ví dụ các điểm trong thành phố), in đường đi và tổng quãng đường ngắn nhất từ một điểm đến mọi điểm còn lại.
- Thử với một đỉnh không thể đến được (khoảng cách vô cùng) và so sánh kết quả với BFS khi mọi trọng số bằng 1.

💡 **Gợi ý:**

- Đẩy vào heap các bộ `(khoảng_cách, đỉnh)`; khi lấy ra mà khoảng cách lớn hơn `dist[u]` đã biết thì bỏ qua (mục heap cũ).
- Bước nới lỏng (relaxation): nếu `dist[u] + w < dist[v]` thì cập nhật `dist[v]` và `parent[v] = u`.
- Độ phức tạp O((V + E) log V) với heap.

💻 **Code mẫu:**

```
import heapq


def dijkstra(graph, source):
    dist = {u: float('inf') for u in graph}
    parent = {u: None for u in graph}
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:  # Mục cũ trong heap, đã có đường tốt hơn
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:  # Nới lỏng cạnh u -> v
                dist[v] = dist[u] + w
                parent[v] = u
                heapq.heappush(heap, (dist[v], v))
    return dist, parent


def get_path(parent, target):
    path = []
    cur = target
    while cur is not None:
        path.append(cur)
        cur = parent[cur]
    return path[::-1]


def build_undirected(edges):
    graph = {}
    for u, v, w in edges:
        graph.setdefault(u, []).append((v, w))
        graph.setdefault(v, []).append((u, w))
    return graph


def main():
    graph = build_undirected([
        ('A', 'B', 4), ('A', 'C', 2), ('B', 'C', 1), ('B', 'D', 5),
        ('C', 'D', 8), ('C', 'E', 10), ('D', 'E', 2), ('D', 'F', 6),
        ('E', 'F', 3),
    ])
    graph['G'] = []  # Đỉnh không thể đến được
```

```
    dist, parent = dijkstra(graph, 'A')
    for v in sorted(graph):
        if dist[v] == float('inf'):
            print(f'A -> {v}: không có đường đi')
        else:
            print(f'A -> {v}: {dist[v]:>2}  đường đi: {" -> ".join(get_path(parent,
v))}')


if __name__ == '__main__':
    main()
```

🤖 **Vận dụng AI để gia tăng trải nghiệm:**

- **Hỏi về lý thuyết:** Hỏi AI: *Vì sao Dijkstra sai khi đồ thị có cạnh âm? Hãy cho một ví dụ cụ thể.*
- **Nâng cao:** Hỏi AI: *Thuật toán A* khác Dijkstra ở điểm nào? Hàm heuristic được chọn như thế nào để vẫn tìm ra đường ngắn nhất?*

Bài tập 8: Cây khung nhỏ nhất – Union-Find và Kruskal – Mức độ: Khó

Cho một mạng gồm nhiều điểm và các đường nối có chi phí, bài toán là nối tất cả các điểm lại với tổng chi phí nhỏ nhất (ví dụ thiết kế mạng cáp quang hoặc đường điện). Lời giải là cây khung nhỏ nhất (MST).

Yêu cầu chi tiết:

- Cài đặt lớp `DisjointSet` (Union-Find) với `find` (nén đường đi) và `union` (trả về `False` nếu hai đỉnh đã cùng tập).
- Cài đặt `kruskal(vertices, edges)` trả về danh sách các cạnh của MST và tổng trọng số.
- Dựng mạng gồm ít nhất 6 đỉnh, 9 cạnh có trọng số; in các cạnh được chọn.
- Xử lý đồ thị không liên thông: khi số cạnh chọn ít hơn V − 1 thì thông báo và trả về rừng khung.
- Kiểm chứng: MST có đúng V − 1 cạnh và nối được mọi đỉnh.

💡 **Gợi ý:**

- Sắp xếp các cạnh theo trọng số tăng dần, duyệt từng cạnh và chỉ nhận cạnh nếu hai đầu mút thuộc hai tập khác nhau (nếu cùng tập thì cạnh đó tạo chu trình).
- Union-Find cho thao tác gần như O(1) khi dùng nén đường đi kết hợp gộp theo hạng (rank).
- Tổng độ phức tạp bị chi phối bởi bước sắp xếp: O(E log E).

💻 **Code mẫu:**

```
class DisjointSet:
    def __init__(self, items):
        self.parent = {x: x for x in items}
        self.rank = {x: 0 for x in items}

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # Nén đường đi
        return self.parent[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
```

```
        if ra == rb:
            return False  # Đã cùng tập -> thêm cạnh sẽ tạo chu trình
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1
        return True


def kruskal(vertices, edges):
    """edges là list các bộ (trọng_số, u, v)."""
    ds = DisjointSet(vertices)
    mst, total = [], 0
    for w, u, v in sorted(edges):
        if ds.union(u, v):
            mst.append((u, v, w))
            total += w
            if len(mst) == len(vertices) - 1:
                break
    return mst, total


def main():
    vertices = ['A', 'B', 'C', 'D', 'E', 'F']
    edges = [
        (7, 'A', 'B'), (5, 'A', 'D'), (8, 'B', 'C'), (9, 'B', 'D'),
        (7, 'B', 'E'), (5, 'C', 'E'), (15, 'D', 'E'), (6, 'D', 'F'),
        (8, 'E', 'F'),
    ]
    mst, total = kruskal(vertices, edges)
    for u, v, w in mst:
        print(f'{u} - {v}: {w}')
    print('Tổng chi phí MST:', total)
    if len(mst) < len(vertices) - 1:
        print('Đồ thị không liên thông: đây chỉ là rừng khung.')
    else:
        print('Số cạnh =', len(mst), '= V - 1:', len(mst) == len(vertices) - 1)


if __name__ == '__main__':
    main()
```

🤖 **Vận dụng AI để gia tăng trải nghiệm:**

- **So sánh:** Hỏi AI: *So sánh Kruskal và Prim: khi nào dùng thuật toán nào (đồ thị thưa so với dày)?*
- **Kiểm thử:** Hỏi AI: *Khi nhiều cạnh có cùng trọng số, MST có duy nhất không? Gợi ý test case cho trường hợp này.*

Bài tập 9: Cạnh âm và chu trình âm – Thuật toán Bellman-Ford – Mức độ: Rất khó

Dijkstra không dùng được khi có cạnh trọng số âm (ví dụ chi phí có thể được hoàn lại). Bellman-Ford xử lý được cạnh âm và còn phát hiện được chu trình âm, tức vòng lặp mà đi càng nhiều vòng thì tổng chi phí càng giảm, khiến đường đi ngắn nhất không tồn tại.

Yêu cầu chi tiết:

- Cài đặt `bellman_ford(vertices, edges, source)` trên danh sách cạnh có hướng `(u, v, w)`.
- Hàm trả về `dist` và `parent` nếu không có chu trình âm; trả về `(None, None)` nếu phát hiện chu trình âm.
- Thử với một đồ thị có cạnh âm nhưng không có chu trình âm, và một đồ thị có chu trình âm.
- Tối ưu hóa: dừng sớm nếu một vòng lặp không cập nhật được khoảng cách nào.
- So sánh kết quả với Dijkstra trên đồ thị không có cạnh âm để kiểm tra tính đúng đắn.

💡 **Gợi ý:**

- Nới lỏng toàn bộ các cạnh V − 1 lần: đường đi ngắn nhất không có chu trình có tối đa V − 1 cạnh.
- Chạy thêm vòng thứ V: nếu vẫn còn cạnh nới lỏng được thì có chu trình âm có thể đến được từ nguồn.
- Chú ý chỉ nới lỏng khi `dist[u]` khác vô cùng để tránh tính toán với vô cùng.
- Độ phức tạp O(V·E), chậm hơn Dijkstra nhưng tổng quát hơn.

💻 **Code mẫu:**

```
INF = float('inf')


def bellman_ford(vertices, edges, source):
    dist = {v: INF for v in vertices}
    parent = {v: None for v in vertices}
    dist[source] = 0
    for _ in range(len(vertices) - 1):
        updated = False
        for u, v, w in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                parent[v] = u
                updated = True
        if not updated:  # Dừng sớm khi đã ổn định
            break
    for u, v, w in edges:  # Vòng kiểm tra chu trình âm
        if dist[u] != INF and dist[u] + w < dist[v]:
            return None, None
    return dist, parent


def main():
    vertices = ['S', 'A', 'B', 'C', 'D']
    ok_edges = [('S', 'A', 4), ('S', 'B', 5), ('A', 'C', -3),
                ('B', 'C', 2), ('C', 'D', 4)]
    dist, _ = bellman_ford(vertices, ok_edges, 'S')
    print('Khoảng cách từ S (cạnh âm, không có chu trình âm):', dist)

    # Chu trình A -> B -> C -> A có tổng 1 + 2 - 5 = -2 < 0
    bad_edges = [('S', 'A', 1), ('A', 'B', 1), ('B', 'C', 2),
                 ('C', 'A', -5), ('C', 'D', 3)]
    dist, _ = bellman_ford(vertices, bad_edges, 'S')
    if dist is None:
```

```
        print('Phát hiện chu trình âm: không tồn tại đường đi ngắn nhất.')


if __name__ == '__main__':
    main()
```

🤖 **Vận dụng AI để gia tăng trải nghiệm:**

- **Hỏi về lý thuyết:** Hỏi AI: *Chứng minh vì sao sau V − 1 vòng nới lỏng thì mọi khoảng cách đã đúng nếu không có chu trình âm?*
- **Thực tế:** Hỏi AI: *Chu trình âm được dùng để phát hiện cơ hội chênh lệch giá (arbitrage) giữa các cặp tiền tệ như thế nào (lấy logarit âm của tỷ giá)?*

Bài tập 10: Cầu và khớp – Đánh giá độ tin cậy của mạng – Mức độ: Rất khó

Trong mạng máy tính hay mạng đường bộ, có những liên kết mà nếu hỏng thì mạng bị chia cắt. Đó là **cầu (bridge)**. Tương tự, **khớp (articulation point)** là đỉnh mà nếu bỏ đi thì đồ thị mất liên thông. Hai khái niệm này dựa trên một kỹ thuật DFS tinh tế gọi là low-link.

Yêu cầu chi tiết:

- Cài đặt `find_bridges_and_articulation_points(graph)` cho đồ thị vô hướng, trả về danh sách cầu và danh sách khớp.
- Dựng mạng mẫu gồm hai nhóm (mỗi nhóm là một tam giác) nối với nhau bằng một cạnh duy nhất; xác nhận cạnh nối là cầu và hai đầu của nó là khớp.
- Viết hàm kiểm chứng `is_connected_without(graph, removed)` bằng cách bỏ thử một đỉnh rồi chạy DFS, đối chiếu với kết quả của thuật toán.
- Thử với đồ thị không có cầu (một chu trình lớn) và đồ thị dạng cây (mọi cạnh đều là cầu).

💡 **Gợi ý:**

- Với mỗi đỉnh lưu `tin[u]` (thời điểm thăm) và `low[u]` (thời điểm thăm nhỏ nhất đạt tới từ cây con của u, kể cả qua một cạnh ngược).
- Cạnh (u, v) với v là con của u là **cầu** nếu `low[v] > tin[u]`: cây con của v không có đường vòng về u hoặc tổ tiên của u.
- Đỉnh u là **khớp** nếu: (a) u không phải gốc và có con v với `low[v] >= tin[u]`, hoặc (b) u là gốc và có từ 2 con trở lên trong cây DFS.
- Độ phức tạp O(V + E).

💻 **Code mẫu:**

```
def find_bridges_and_articulation_points(graph):
    tin, low = {}, {}
    timer = [0]
    bridges, points = [], set()

    def dfs(u, parent):
        tin[u] = low[u] = timer[0]
        timer[0] += 1
        children = 0
        for v in graph[u]:
```

```
            if v == parent:
                continue
            if v in tin:  # Cạnh ngược
                low[u] = min(low[u], tin[v])
            else:
                dfs(v, u)
                low[u] = min(low[u], low[v])
                children += 1
                if low[v] > tin[u]:
                    bridges.append((u, v))
                if parent is not None and low[v] >= tin[u]:
                    points.add(u)
        if parent is None and children > 1:
            points.add(u)

    for s in graph:
        if s not in tin:
            dfs(s, None)
    return bridges, sorted(points)


def is_connected_without(graph, removed):
    rest = [u for u in graph if u != removed]
    if not rest:
        return True
    visited, stack = {rest[0]}, [rest[0]]
    while stack:
        u = stack.pop()
        for v in graph[u]:
            if v != removed and v not in visited:
                visited.add(v)
                stack.append(v)
    return len(visited) == len(rest)


def build_undirected(edges):
    graph = {}
    for u, v in edges:
        graph.setdefault(u, []).append(v)
        graph.setdefault(v, []).append(u)
    return graph


def main():
    # Hai tam giác ABC và DEF nối với nhau bằng cạnh C-D
    graph = build_undirected([
        ('A', 'B'), ('B', 'C'), ('C', 'A'),
        ('C', 'D'),
        ('D', 'E'), ('E', 'F'), ('F', 'D'),
    ])
    bridges, points = find_bridges_and_articulation_points(graph)
    print('Cầu  :', bridges)  # [('C', 'D')]
    print('Khớp :', points)  # ['C', 'D']
    for u in sorted(graph):
        print(f'Bỏ {u}: còn liên thông? {is_connected_without(graph, u)}')
```

```
    ring = build_undirected([('1', '2'), ('2', '3'), ('3', '4'), ('4', '1')])
    print('Chu trình lớn:', find_bridges_and_articulation_points(ring))  # Không có cầu, không có khớp


if __name__ == '__main__':
    main()
```

🤖 **Vận dụng AI để gia tăng trải nghiệm:**

- **Hỏi về lý thuyết:** Hỏi AI: *Giải thích trực quan ý nghĩa của mảng low-link. Vì sao điều kiện của cầu dùng dấu > còn điều kiện của khớp dùng dấu >=?*
- **Mở rộng:** Hỏi AI: *Thuật toán Tarjan tìm thành phần liên thông mạnh trong đồ thị có hướng có điểm gì giống với thuật toán low-link ở bài này?*